

Email Communication Simulation System

SMTP-Based Socket Programming Implementation

Introduction to Computer Networks (CS3004301)

Fall Semester 2025

Team Members:

Gerald M. Pillian (F11415009)

Deborah Dobles (F11415012)

Fenja Wiechmann (F11415004)

Quentin Dufour (F11415003)

Guillaume Siossian (F11415002)

Instructor: Distinguished Professor Huei-Wen Ferng

Department of Computer Science and Information Engineering
National Taiwan University of Science and Technology

December 17, 2025

Contents

1	Introduction	3
1.1	Project Objectives	3
1.2	System Architecture Overview	4
2	Technical Background and Protocol Analysis	5
2.1	SMTP Protocol Fundamentals	5
2.2	Socket Programming Considerations	5
2.3	Authentication Mechanisms	6
3	System Design and Implementation	6
3.1	Multi-Threaded SMTP Server Architecture	6
3.2	Message Queue and Delivery System	7
3.3	Client Application Design	8
3.4	Advanced Features	8
3.4.1	Extended SMTP (ESMTP) Support	8
3.4.2	Comprehensive Logging System	8
3.4.3	Connection Rate Limiting	9
4	Implementation Details and Code Structure	9
4.1	Server Implementation Components	9
4.2	Key Algorithms and Data Structures	9
4.2.1	Command Parsing Algorithm	9
4.2.2	Message Queue Data Structure	10
4.3	Threading and Synchronization	10
5	Testing and Validation	11
5.1	Unit Testing	11
5.2	Integration Testing	11
5.3	Performance Evaluation	12

6	Results and Demonstration	12
6.1	System Functionality	12
6.2	Protocol Compliance	13
6.3	Demonstration Scenarios	13
7	Challenges and Solutions	13
7.1	Threading Complexity	13
7.2	Protocol State Management	14
7.3	Error Recovery	14
7.4	Message Persistence	14
8	Future Enhancements	14
9	Conclusion	15
10	Individual Contributions	16
11	References	16

1 Introduction

Email communication remains one of the most fundamental applications of computer networks, serving billions of users worldwide. The Simple Mail Transfer Protocol (SMTP) forms the backbone of email transmission across the Internet, operating as a text-based application-layer protocol that enables reliable message delivery between mail servers and clients.

This project implements an advanced Email Communication Simulation System that demonstrates the complete lifecycle of email transmission using socket programming. Our system extends beyond basic SMTP functionality by incorporating authentication mechanisms, multi-threaded server architecture, message queuing systems, and comprehensive error handling protocols. The implementation provides practical insights into transport-layer socket programming, application-layer protocol design, and concurrent network service management.

The motivation behind this project stems from the need to understand the underlying mechanisms that power everyday email services. While modern email clients abstract away the complexity of message transmission, our simulation exposes the intricate handshake procedures, data encoding schemes, and reliability mechanisms that ensure successful email delivery across heterogeneous networks.

1.1 Project Objectives

The primary objectives of this project are:

- Implement a fully functional SMTP server capable of handling multiple concurrent client connections using multi-threaded architecture
- Develop an SMTP client that supports message composition, authentication, and transmission with proper error handling
- Design and implement a message queuing system for asynchronous email processing and delivery

- Incorporate SMTP authentication (AUTH) mechanisms to simulate secure email transmission
- Implement comprehensive logging and monitoring systems to track message flow and system performance
- Demonstrate protocol extensibility through custom SMTP extensions

1.2 System Architecture Overview

Our Email Communication Simulation System follows a client-server architecture with the following key components:

1. **SMTP Server Module:** A multi-threaded server that listens for incoming connections, processes SMTP commands, and manages message storage
2. **SMTP Client Module:** An interactive client application that allows users to compose and send emails through the socket interface
3. **Message Queue System:** A persistent storage mechanism that queues outgoing messages and implements retry logic for failed deliveries
4. **Authentication Module:** Implements BASE64-encoded authentication following RFC 4954 specifications
5. **Mail Storage System:** A file-based mailbox system that stores received messages in a structured format
6. **Logging and Monitoring:** Comprehensive logging system that tracks all SMTP transactions and system events

2 Technical Background and Protocol Analysis

2.1 SMTP Protocol Fundamentals

The Simple Mail Transfer Protocol operates on TCP port 25 (or 587 for submission) and follows a command-response paradigm. The protocol defines a series of text-based commands that clients send to servers, with each command eliciting a three-digit numeric response code followed by optional human-readable text.

The standard SMTP transaction follows this sequence:

1. Connection establishment and server greeting (220 response)
2. Client identification using EHLO/HELO command
3. Optional authentication (AUTH command)
4. Sender specification (MAIL FROM command)
5. Recipient specification (RCPT TO command)
6. Message data transmission (DATA command)
7. Connection termination (QUIT command)

Each command must be acknowledged by the server with appropriate response codes. Success responses begin with 2xx codes, while 4xx codes indicate temporary failures and 5xx codes indicate permanent failures.

2.2 Socket Programming Considerations

Our implementation utilizes Python's socket library to establish TCP connections between clients and servers. The key technical considerations include:

Connection Management: TCP provides reliable, connection-oriented communication. Our server must handle the three-way handshake implicitly through the `accept()` method, while clients initiate connections using `connect()`.

Data Encoding: SMTP requires ASCII text transmission. All data exchanged between client and server must be encoded to bytes using UTF-8 encoding before transmission and decoded upon reception.

Buffer Management: Network data arrives in chunks. Our implementation uses appropriate buffer sizes (typically 4096 bytes) and implements proper message boundary detection using CRLF (Carriage Return Line Feed) delimiters.

Concurrency: To handle multiple simultaneous client connections, our server implements threading, where each client connection spawns a new thread to handle that client's session independently.

2.3 Authentication Mechanisms

Our system implements SMTP AUTH using the PLAIN mechanism as specified in RFC 4954. The authentication process involves:

1. Client sends AUTH PLAIN command
2. Server responds with 334 response code requesting credentials
3. Client sends BASE64-encoded string containing: `[authzid]\0[authcid]\0[password]`
4. Server validates credentials and responds with 235 (success) or 535 (failure)

The BASE64 encoding ensures that special characters in passwords can be safely transmitted over the ASCII-based SMTP protocol.

3 System Design and Implementation

3.1 Multi-Threaded SMTP Server Architecture

The SMTP server represents the core component of our system, designed to handle multiple concurrent client connections efficiently. The server architecture consists of several layers:

Main Server Loop: The primary thread continuously listens on the designated port (default 2525 for our simulation) using a socket in listening mode. Upon accepting a connection, it spawns a new thread to handle that client session.

Client Handler Threads: Each client connection executes in its own thread, maintaining session state throughout the SMTP transaction. This design allows the server to process multiple email submissions simultaneously without blocking.

State Machine Implementation: Each client handler implements a state machine that tracks the progression through SMTP states: INITIAL, EHLO_RECEIVED, AUTH_REQUIRED, AUTH_COMPLETED, MAIL_FROM_RECEIVED, RCPT_TO_RECEIVED, DATA_RECEIVED, and QUIT_RECEIVED.

The state machine ensures that commands are only accepted in valid sequences. For example, the DATA command can only be executed after at least one RCPT TO command has been successfully processed.

3.2 Message Queue and Delivery System

To simulate realistic email behavior, our system implements a message queue that decouples message acceptance from message delivery. The queue system provides:

Asynchronous Processing: Messages accepted by the SMTP server are immediately queued, allowing the server to respond quickly to clients while processing occurs in the background.

Delivery Retry Logic: Failed delivery attempts trigger exponential backoff retry mechanisms. The system attempts redelivery at increasing intervals (1 min, 5 min, 15 min, 1 hour, 4 hours) before marking messages as undeliverable.

Priority Queuing: Messages can be assigned priority levels, with high-priority messages processed before lower-priority ones in the queue.

Persistent Storage: The queue persists to disk using JSON serialization, ensuring that queued messages survive server restarts.

3.3 Client Application Design

The SMTP client provides an interactive interface for composing and sending emails.

Key features include:

Interactive Composition: Users input sender address, recipient addresses (supporting multiple recipients), subject line, and message body through a command-line interface.

Protocol Implementation: The client handles the complete SMTP transaction sequence, including authentication, proper command formatting, and response parsing.

Error Handling: Comprehensive error handling detects network failures, authentication errors, invalid recipient addresses, and server rejections, providing meaningful feedback to users.

MIME Support: The client supports basic MIME header generation for proper email formatting, including Date, From, To, Subject, and Message-ID headers.

3.4 Advanced Features

3.4.1 Extended SMTP (ESMTP) Support

Our server implements Extended SMTP features announced through the EHLO response:

- **SIZE:** Declares maximum message size (10 MB in our implementation)
- **AUTH PLAIN:** Advertises authentication support
- **PIPELINING:** Allows clients to send multiple commands without waiting for responses
- **8BITMIME:** Supports 8-bit MIME content

3.4.2 Comprehensive Logging System

Every SMTP transaction is logged with timestamps, client IP addresses, command sequences, and outcomes. The logging system uses Python's logging module with both

file and console handlers, supporting different log levels (DEBUG, INFO, WARNING, ERROR) for development and production environments.

3.4.3 Connection Rate Limiting

To prevent abuse, the server implements connection rate limiting that restricts the number of connections from a single IP address within a time window, simulating anti-spam measures used by production mail servers.

4 Implementation Details and Code Structure

4.1 Server Implementation Components

The server implementation is organized into modular components:

SMTPServer Class: Main server class that initializes the listening socket, manages the thread pool, and coordinates overall server operation.

ClientHandler Class: Manages individual client sessions, implementing the SMTP state machine and command processing logic.

MessageQueue Class: Implements the priority queue with persistent storage and retry logic.

Authenticator Class: Handles credential validation against a user database (implemented as a JSON file for our simulation).

MailStorage Class: Manages the mailbox system, storing received messages in individual files organized by recipient.

4.2 Key Algorithms and Data Structures

4.2.1 Command Parsing Algorithm

The command parsing algorithm uses regular expressions to extract commands and parameters from client input. The algorithm validates command syntax and parameters before state transitions:

Algorithm 1 SMTP Command Processing

```

1: procedure PROCESSCOMMAND(command_line)
2:   parts  $\leftarrow$  SPLIT(command_line)
3:   cmd  $\leftarrow$  UPPERCASE(parts[0])
4:   if cmd  $\in$  VALID_COMMANDS then
5:     handler  $\leftarrow$  GET_HANDLER(cmd)
6:     if VALIDATE_STATE(cmd, current_state) then
7:       response  $\leftarrow$  handler(parts[1 :])
8:       UPDATE_STATE(cmd)
9:       return response
10:    else
11:      return "503 Bad sequence of commands"
12:    end if
13:  else
14:    return "500 Command not recognized"
15:  end if
16: end procedure

```

4.2.2 Message Queue Data Structure

The message queue uses a priority heap (min-heap) where each element contains:

- Priority level (1-5, lower is higher priority)
- Timestamp of queue insertion
- Number of delivery attempts
- Sender and recipient information
- Message content
- Next retry time (for failed deliveries)

The heap property ensures that highest-priority messages with the earliest next-retry times are processed first.

4.3 Threading and Synchronization

The multi-threaded architecture requires careful synchronization to prevent race conditions:

Message Queue Access: A threading.Lock protects queue operations, ensuring that only one thread modifies the queue at a time.

Mailbox Access: File-level locking using fcntl (on Unix systems) prevents simultaneous writes to the same mailbox file.

Connection Counting: A thread-safe counter tracks active connections, allowing the server to enforce connection limits.

5 Testing and Validation

5.1 Unit Testing

We developed comprehensive unit tests for each component:

- **Protocol Compliance Tests:** Verify correct handling of all SMTP commands and response codes
- **Authentication Tests:** Validate correct BASE64 encoding/decoding and credential verification
- **State Machine Tests:** Ensure commands are only accepted in valid states
- **Queue Operations Tests:** Verify message enqueueing, dequeueing, and priority ordering
- **Error Handling Tests:** Confirm appropriate responses to malformed commands and network errors

5.2 Integration Testing

Integration tests validate the complete email transmission workflow:

1. Client connects to server and receives greeting
2. Client authenticates successfully

3. Client sends email to valid recipient
4. Server queues message
5. Queue processor delivers message to mailbox
6. Verify message content and headers in mailbox

We also test failure scenarios, including authentication failures, invalid recipients, oversized messages, and network disconnections during transmission.

5.3 Performance Evaluation

We conducted performance tests to evaluate system scalability:

Concurrent Connection Test: Measured server performance with 100, 500, and 1000 simultaneous client connections. The server maintained stable response times up to 500 concurrent connections before thread exhaustion became a limiting factor.

Message Throughput Test: Evaluated messages processed per second, achieving approximately 50 messages/second on our test hardware (Intel Core i5, 8GB RAM).

Queue Processing Latency: Measured time from message acceptance to mailbox delivery, averaging 100-200ms under normal load conditions.

6 Results and Demonstration

6.1 System Functionality

Our Email Communication Simulation System successfully demonstrates all core functionalities:

- Successful SMTP transactions between multiple clients and the server
- Proper authentication and session management
- Reliable message queuing and delivery

- Appropriate error handling for various failure scenarios
- Comprehensive logging of all system activities

6.2 Protocol Compliance

The system adheres to SMTP specifications defined in RFC 5321 and RFC 4954 (SMTP AUTH). Command syntax, response codes, and transaction sequences all conform to standard specifications, ensuring interoperability with standard SMTP clients and servers.

6.3 Demonstration Scenarios

During the oral presentation, we will demonstrate:

1. **Basic Email Transmission:** A client successfully sends an email through our server
2. **Multiple Recipients:** Sending a single message to multiple recipients
3. **Authentication Failure:** Handling incorrect credentials
4. **Invalid Recipient:** Server rejection of non-existent recipient addresses
5. **Concurrent Connections:** Multiple clients simultaneously connecting and sending emails
6. **Queue Management:** Viewing queued messages and retry logic in action
7. **Live Code Modification:** Adapting the code to implement minor enhancements as requested

7 Challenges and Solutions

7.1 Threading Complexity

Challenge: Managing thread synchronization and preventing race conditions in shared resources.

Solution: Implemented comprehensive locking mechanisms using Python's threading primitives (Lock, RLock) and carefully designed critical sections to minimize lock contention.

7.2 Protocol State Management

Challenge: Ensuring proper SMTP command sequencing and state transitions.

Solution: Designed a formal state machine with explicit state definitions and transition rules, validated through extensive testing.

7.3 Error Recovery

Challenge: Handling network failures, client disconnections, and incomplete transactions.

Solution: Implemented try-except blocks around all socket operations, timeout mechanisms for stalled connections, and transaction rollback for incomplete message submissions.

7.4 Message Persistence

Challenge: Ensuring queued messages survive server restarts without corruption.

Solution: Implemented atomic file operations using temporary files and rename operations, combined with regular queue serialization to persistent storage.

8 Future Enhancements

While our current implementation provides robust SMTP functionality, several enhancements could further improve the system:

TLS/SSL Support: Implementing STARTTLS command to encrypt SMTP sessions, protecting credentials and message content.

DKIM Signing: Adding DomainKeys Identified Mail signatures to authenticate message origins and prevent spoofing.

SPF Validation: Implementing Sender Policy Framework checks to validate sender authenticity.

Advanced Queue Management: Implementing message priority classes, delivery scheduling, and more sophisticated retry algorithms.

Web-Based Management Interface: Developing a web UI for monitoring server status, viewing logs, and managing queued messages.

POP3/IMAP Server: Implementing message retrieval protocols to allow clients to access received messages.

Database Integration: Replacing file-based storage with a relational database for improved scalability and query capabilities.

9 Conclusion

This project successfully demonstrates the implementation of a comprehensive Email Communication Simulation System using socket programming and the SMTP protocol. Through this implementation, we gained deep insights into application-layer protocol design, transport-layer socket operations, and the challenges of building reliable network services.

The multi-threaded server architecture efficiently handles concurrent connections while maintaining protocol compliance and state consistency. The message queue system provides asynchronous processing with retry logic, simulating real-world email infrastructure behavior. Authentication mechanisms ensure secure access, while comprehensive logging enables system monitoring and debugging.

Beyond technical implementation, this project enhanced our understanding of how fundamental Internet services operate at the protocol level. The experience of designing, implementing, testing, and debugging a network application provides invaluable practical knowledge that complements the theoretical concepts covered in the course curriculum.

The system's modular design and extensible architecture provide a solid foundation for future enhancements, demonstrating not only current functionality but also principles

of sustainable software engineering applicable to production network services.

10 Individual Contributions

- **Gerald M. Pillian (F11415009):** SMTP server core implementation, state machine design, protocol compliance, and file operations
- **Fenja Wiechmann (F11415004):** Client application development, user interface design, command-line parsing
- **Quentin Dufour (F11415003):** Authentication module, BASE64 encoding/decoding, credential management, mail storage system
- **Guillaume Siossian (F11415002):** Message queue system, priority heap implementation, retry logic
- **Deborah Dobles (F11415012):** Logging system, testing framework, documentation, integration testing, and mailbox management

11 References

1. Kurose, J. F., & Ross, K. W. (2021). *Computer Networking: A Top-Down Approach* (8th ed.). Pearson.
2. Klensin, J. (2008). Simple Mail Transfer Protocol. RFC 5321. Internet Engineering Task Force. <https://tools.ietf.org/html/rfc5321>
3. Siemborski, R., & Melnikov, A. (2007). SMTP Service Extension for Authentication. RFC 4954. Internet Engineering Task Force. <https://tools.ietf.org/html/rfc4954>
4. Python Software Foundation. (2024). Socket Programming HOWTO. <https://docs.python.org/3/howto/sockets.html>

5. Python Software Foundation. (2024). Threading — Thread-based parallelism.
<https://docs.python.org/3/library/threading.html>
6. Crispin, M. (2003). Internet Message Access Protocol - Version 4rev1. RFC 3501.
Internet Engineering Task Force. <https://tools.ietf.org/html/rfc3501>
7. Myers, J., & Rose, M. (1996). Post Office Protocol - Version 3. RFC 1939. Internet
Engineering Task Force. <https://tools.ietf.org/html/rfc1939>
8. Freed, N., & Borenstein, N. (1996). Multipurpose Internet Mail Extensions (MIME)
Part One. RFC 2045. Internet Engineering Task Force. <https://tools.ietf.org/html/rfc2045>